

Requirements Engineering

UniMentor / Software Engineering · Stakeholders, business rules, comprehensive use cases and non-functional requirements.

1. Stakeholder Analysis and Primary Concerns

Beyond simply listing roles, we identify each stakeholder's operational focus, friction points, and acceptance criteria within the platform:

Stakeholder	System Role	Primary Concerns	Acceptance Criteria
Student (Mentee)	End user seeking academic support and course-specific tutoring.	Speed, usability, and schedule certainty: Needs to quickly locate verified mentors by subject, book slots with minimal friction, receive prompt confirmations, and ensure sessions never overlap or get canceled without notice.	Complete a booking flow in ≤ 3 minutes (median) and ≤ 6 clicks (G1); receive the confirmation email within 60 seconds in $\geq 95\%$ of bookings (G5).
Tutor (Mentor)	Knowledge provider offering academic sessions and managing availability.	Schedule control and zero collisions: Demands strict control over availability, the ability to cancel with fair advance notice, zero schedule collisions, and early access to the student's inquiry context.	Zero double bookings at the database level; intuitive slot management and predictable booking windows.
University Administrator (Admin)	Platform governance, user verification, and compliance supervisor.	Security, governance, and auditability: Focuses on institutional identity validation, GDPR-compliant data processing, session traceability, and immediate enforcement actions against misbehaving accounts.	Centralized dashboard featuring immutable activity logs, role-based access control (RBAC), and fast account suspension workflows.

2. Global Business Rules and Parameter Constraints

To prevent ambiguity during implementation, the team has formally finalized all functional limits and operational constraints:

- **BR-CONF-01 (Booking Advance Notice):** Tutoring sessions must be booked at least **2 hours prior** to the slot start time. Same-day immediate bookings within this 2-hour window are strictly rejected to allow preparation time.
 - **BR-CONF-02 (Cancellation Policy):** Both students and tutors can cancel an appointment without penalty up to **4 hours before** the session starts. Late cancellations are flagged in the user profile.
 - **BR-CONF-03 (Standard Session Duration):** All tutoring sessions are scheduled in fixed **60-minute** blocks. The session type chosen at booking (Quick Check, Standard Tutoring or Deep Dive, see Section 1) sets the intended maximum length of the conversation inside the booked block; it does not change the block size.
 - **BR-CONF-04 (Active Booking Limit):** A student may hold a maximum of **3 active pending appointments** concurrently to avoid slot hoarding.
 - **BR-CONF-05 (Strict Collision Prevention):** Overlapping sessions for either a tutor or a student are forbidden and enforced through atomic database constraints.
-

3. Comprehensive Use Case Catalog

CU-01: User Registration

- **ID:** CU-01
- **Name:** Account Registration
- **Primary Actor:** Unregistered User (Student or Tutor)
- **Secondary Actors:** Database (PostgreSQL), Notification Service (SendGrid)
- **Trigger:** The user clicks "Create Account" on the registration form.
- **Preconditions:**
 1. The user has no active session.
 2. The email address belongs to an authorized university domain.
- **Postconditions:**
 - **Success:** The user record is created with a bcrypt-hashed password, the selected role is assigned, and a welcome confirmation email is sent.
 - **Failure:** No data is stored; input validation feedback is rendered on the form.
- **Business Rules:** Institutional email must be unique across the platform; passwords must be at least 8 characters long and contain both letters and digits.
- **Main Success Scenario (Happy Path):**
 1. The user navigates to the registration screen.

2. The user submits full name, institutional email, password, and designated role (Student or Tutor).
 3. The user clicks "Create Account".
 4. The backend checks payload formats and field integrity.
 5. The backend verifies that the email address is not already registered.
 6. The system generates a cryptographic password hash using bcrypt.
 7. The record is persisted in the database with the chosen role.
 8. An automated welcome email is dispatched.
 9. The user is redirected to the login interface with a success toast notification.
- **Alternative and Error Flows:**
 - **A1: Duplicate Email:** In step 5, if the email already exists in PostgreSQL, the server returns an HTTP 409 Conflict. The form highlights: *"This email address is already in use."*
 - **A2: Weak Password:** In step 4, if the password fails strength criteria, the submission is rejected with an HTTP 400 Bad Request, highlighting missing requirements.
-

CU-02: User Authentication

- **ID:** CU-02
- **Name:** Account Login and Session Creation
- **Primary Actor:** Registered User (Student, Tutor, or Admin)
- **Secondary Actors:** Database (PostgreSQL)
- **Trigger:** The user submits credentials by clicking "Sign In".
- **Preconditions:** The user account exists and remains in active status.
- **Postconditions:**
 - **Success:** A signed JSON Web Token (JWT) is issued, and the user is redirected to their respective role-based dashboard.
 - **Failure:** Authentication is denied, and the failed login counter increases.
- **Business Rules:** After 5 consecutive failed login attempts, the account is temporarily locked for 15 minutes.
- **Main Success Scenario (Happy Path):**
 1. The user inputs email and password on the login screen and submits.
 2. The backend retrieves the corresponding record from PostgreSQL.
 3. The system matches the plaintext password against the stored bcrypt hash.
 4. Upon validation, the server generates a JWT containing the user ID and role claims.
 5. The token is returned in a **Set-Cookie** header as an **HttpOnly**, **Secure** cookie (ADR 4); client-side JavaScript never has access to it.
 6. The frontend updates authentication state and loads the role-specific dashboard.
- **Alternative and Error Flows:**
 - **A1: Invalid Credentials:** If the email or password hash fails validation, the API returns an HTTP 401 Unauthorized: *"Invalid email or password."*

- **A2: Account Suspended:** If the account is marked as suspended, authentication is halted with an HTTP 403 Forbidden: *"Your account has been suspended. Please contact administration."*
 - **A3: Account Locked:** After 5 consecutive failed attempts, further logins during the next 15 minutes are rejected with an HTTP 429 Too Many Requests: *"Too many failed attempts. Try again in 15 minutes."*
-

CU-03: Tutor Profile and Subject Setup

- **ID:** CU-03
 - **Name:** Manage Tutor Academic Profile and Subjects
 - **Primary Actor:** Tutor
 - **Secondary Actors:** Database (PostgreSQL)
 - **Trigger:** The tutor clicks "Save Profile".
 - **Preconditions:** The user is authenticated with the Tutor role.
 - **Postconditions:**
 - **Success:** The tutor's profile details, assigned subjects, bio, and recurring meeting link are updated in the database.
 - **Failure:** Existing profile data remains untouched, and a validation alert is shown.
 - **Business Rules:** At least one authorized subject must be selected; the meeting link must be a valid URL (Google Meet, Zoom, or Microsoft Teams).
 - **Main Success Scenario (Happy Path):**
 1. The tutor opens the "My Profile" tab.
 2. The interface fetches current profile parameters and the university subject catalog.
 3. The tutor edits biographical details, selects supported subjects, and inputs their recurring virtual meeting link.
 4. The tutor submits the changes.
 5. The backend validates URL format and verifies that at least one subject association exists.
 6. Database records in `tutor_profiles` and `tutor_subjects` are updated.
 7. A confirmation toast confirms: *"Profile successfully updated."*
 - **Alternative and Error Flows:**
 - **A1: Missing Subject Selection:** If no subjects are selected upon submission, the backend rejects the update, prompting the user to select at least one course.
 - **A2: Invalid Meeting URL:** If the meeting link is malformed, validation fails with an HTTP 400 Bad Request.
-

CU-04: Publish Availability Slots (Tutor)

- **ID:** CU-04

- **Name:** Create Open Calendar Slots
 - **Primary Actor:** Tutor
 - **Secondary Actors:** Database (PostgreSQL)
 - **Trigger:** The tutor clicks "Publish Slots" in the calendar view.
 - **Preconditions:** The tutor must have an active profile with at least one linked subject.
 - **Postconditions:**
 - **Success:** Availability slots are persisted with **AVAILABLE** status and become bookable for students.
 - **Failure:** Conflicting or historical slots are rejected.
 - **Business Rules:** Slots are standardized at 60 minutes; slots cannot be created in the past or overlap existing slots belonging to the same tutor.
 - **Main Success Scenario (Happy Path):**
 1. The tutor accesses their availability management calendar.
 2. The tutor selects open time intervals on the weekly grid.
 3. The system partitions the selection into distinct 60-minute blocks.
 4. The tutor clicks "Publish Slots".
 5. The server checks for collisions against existing slots and scheduled appointments.
 6. Valid blocks are inserted into **availability_slots** with **AVAILABLE** status.
 7. The calendar interface updates to display the new slots in green.
 - **Alternative and Error Flows:**
 - **A1: Slot Overlap Detected:** If any selected block conflicts with an existing slot, the backend skips the overlapping range, persists non-conflicting slots, and notifies the tutor about the discarded segment.
-

CU-05: Remove or Block Availability Slot (Tutor)

- **ID:** CU-05
- **Name:** Remove Calendar Slot
- **Primary Actor:** Tutor
- **Secondary Actors:** Database (PostgreSQL)
- **Trigger:** The tutor clicks on an available slot and selects "Delete Slot".
- **Preconditions:** The slot belongs to the authenticated tutor.
- **Postconditions:**
 - **Success:** The slot is deleted or marked as **BLOCKED**, removing it from the public booking view.
 - **Failure:** The slot remains active, and an explanatory warning is returned.
- **Business Rules:** A slot cannot be deleted directly if it is already linked to a confirmed appointment; confirmed sessions must follow the formal cancellation workflow (CU-09).
- **Main Success Scenario (Happy Path):**
 1. The tutor selects an open slot on their calendar grid.
 2. The tutor clicks "Delete Slot" and confirms the modal dialog.

3. The server validates that the slot is in **AVAILABLE** status.
 4. The record is deleted or updated to **BLOCKED** in **availability_slots**.
 5. The calendar updates immediately.
- **Alternative and Error Flows:**
 - **A1: Slot Concurrently Booked:** If a student booked the slot right before deletion, the server identifies its **BOOKED** status, rejects direct deletion, and returns an HTTP 409 Conflict: *"This slot has just been booked. Please cancel it from your appointments dashboard."*
-

CU-06: Search and Filter Tutors

- **ID:** CU-06
 - **Name:** Browse and Filter Academic Mentors
 - **Primary Actor:** Student
 - **Secondary Actors:** Database (PostgreSQL)
 - **Trigger:** The student enters query text or selects a subject from the dropdown filter.
 - **Preconditions:** None.
 - **Postconditions:**
 - **Success:** A list of matching active tutors with their expertise and upcoming availability is displayed.
 - **Failure:** A zero-results message is shown.
 - **Business Rules:** Only verified tutors with active status and future availability are prioritized in the directory.
 - **Main Success Scenario (Happy Path):**
 1. The student navigates to the "Find Tutors" page.
 2. The student filters by subject (e.g., "Software Engineering").
 3. The API executes a parameterized search across **tutor_profiles** and **subjects**.
 4. The backend responds in under 200 ms (adhering to NFR-01).
 5. The UI renders mentor profile cards with upcoming availability highlights.
 - **Alternative and Error Flows:**
 - **A1: No Matches Found:** If no tutors match the search criteria, the view displays: *"No mentors available for the selected subject at this time."*
-

CU-07: Inspect Tutor Calendar

- **ID:** CU-07
- **Name:** View Tutor Real-Time Schedule
- **Primary Actor:** Student
- **Secondary Actors:** Database (PostgreSQL)
- **Trigger:** The student clicks "View Availability" on a tutor's card.

- **Preconditions:** The selected tutor exists and holds active status.
 - **Postconditions:**
 - **Success:** An interactive calendar renders selectable open slots.
 - **Failure:** The view indicates no upcoming slots are available.
 - **Business Rules:** Slots scheduled within the next 2 hours are disabled to enforce the minimum advance booking rule (BR-CONF-01).
 - **Main Success Scenario (Happy Path):**
 1. The student opens the tutor profile and selects "View Availability".
 2. The client fetches calendar data via `GET /api/tutors/:id/availability`.
 3. The backend filters for `AVAILABLE` slots where start time exceeds the current timestamp plus 2 hours.
 4. The frontend renders open slots highlighted in green.
 - **Alternative and Error Flows:**
 - **A1: No Published Slots:** If the tutor has not configured slots for the period, the calendar displays: *"This tutor has no available sessions published for this period."*
-

CU-08: Book Tutoring Session

- **ID:** CU-08
- **Name:** Schedule a Tutoring Appointment
- **Primary Actor:** Student
- **Secondary Actors:** Tutor, Database (PostgreSQL), Notification Service (SendGrid)
- **Trigger:** The student clicks "Confirm Booking" on a selected slot.
- **Preconditions:**
 1. The student is authenticated with an active session.
 2. The selected slot is in `AVAILABLE` status.
 3. The student has fewer than 3 active pending bookings.
- **Postconditions:**
 - **Success:** An appointment is created in `CONFIRMED` status, the slot shifts to `BOOKED`, and notification emails containing meeting links are dispatched to both parties.
 - **Failure:** The slot remains unchanged, the database transaction is rolled back, and the user receives an error notice.
- **Business Rules:** Minimum 2-hour booking advance notice; no overlapping appointments for the student; maximum 3 active appointments per student.
- **Main Success Scenario (Happy Path):**
 1. The student selects an open slot on the tutor's calendar.
 2. The student chooses a session type (Quick Check, Standard Tutoring or Deep Dive) and inputs a brief description of their academic inquiry.
 3. The student clicks "Confirm Booking".
 4. The backend opens an isolated PostgreSQL transaction (`BEGIN`).

5. The server checks the advance notice window, user collision constraints, and the active booking quota.
 6. The slot status updates to **BOOKED**, and a new record (including the session type) is created in **appointments**.
 7. The transaction commits successfully (**COMMIT**).
 8. Asynchronous notification emails containing session details and meeting links are dispatched.
 9. The user is redirected to their appointments dashboard with a success confirmation.
- **Alternative and Error Flows:**
 - **A1: Race Condition (Concurrent Booking):** If another student books the same slot simultaneously, the transaction detects the status change, triggers a **ROLLBACK**, and returns an HTTP 409 Conflict: *"This time slot has just been reserved by another student."*
 - **A2: Quota Exceeded:** If the student already has 3 pending bookings, the system rejects the booking with an HTTP 400 Bad Request: *"You have reached the maximum limit of 3 active appointments."*
 - **A3: Insufficient Advance Notice:** If the selected slot starts in less than 2 hours, the booking is rejected: *"Reservations require at least 2 hours advance notice."*
-

CU-09: Cancel Tutoring Session

- **ID:** CU-09
- **Name:** Cancel Scheduled Appointment
- **Primary Actor:** Student or Tutor
- **Secondary Actors:** Counterparty User, Database, Notification Service
- **Trigger:** The user clicks "Cancel Session" from their dashboard appointment list.
- **Preconditions:** The appointment is in **CONFIRMED** status, and the user is an authorized participant.
- **Postconditions:**
 - **Success:** The session status changes to **CANCELLED**, the slot reverts to **AVAILABLE** (if within policy), and cancellation emails are dispatched.
 - **Failure:** The appointment remains active, and an error message is shown.
- **Business Rules:** Penalty-free cancellation requires at least 4 hours advance notice (BR-CONF-02).
- **Main Success Scenario (Happy Path):**
 1. The user navigates to "Upcoming Sessions" on their dashboard.
 2. The user clicks "Cancel Session" on the target appointment.
 3. The user provides an optional cancellation reason and confirms.
 4. The server validates that the session start time is more than 4 hours away.
 5. The appointment updates to **CANCELLED**, and the slot reverts to **AVAILABLE**.
 6. Automated cancellation notices are emailed to both users.

7. The appointment is moved to the past/cancelled tab in the dashboard.

- **Alternative and Error Flows:**
 - **A1: Late Cancellation (< 4 Hours):** If the cancellation occurs under 4 hours before the session, the system warns of policy non-compliance, logs the late cancellation on the profile, and keeps the slot closed to prevent sudden scheduling gaps.
-

CU-10: Automated Reminders Dispatch

- **ID:** CU-10
 - **Name:** Automatic Email Reminder Service
 - **Primary Actor:** Background Service (Node.js Scheduler / Cron)
 - **Secondary Actors:** Email Provider (SendGrid API), Student, Tutor
 - **Trigger:** Periodic backend cron job executing every 10 minutes.
 - **Preconditions:** Confirmed appointments scheduled within the next 24 hours exist without a sent reminder flag.
 - **Postconditions:**
 - **Success:** Reminder emails containing session time and meeting URLs are sent to both participants.
 - **Failure:** Failed attempts are logged, and records remain queued for retry.
 - **Business Rules:** Reminders are dispatched 24 hours prior to appointment start time (tolerance $\pm$ 15 minutes, G5; the 10-minute scheduler period keeps dispatch inside that window). Appointments booked less than 24 hours ahead receive their reminder in the next scheduler cycle.
 - **Main Success Scenario (Happy Path):**
 1. The scheduler queries PostgreSQL for confirmed appointments occurring within the 24-hour window where `reminder_sent = false`.
 2. The service renders email templates with booking details and the meeting link.
 3. The service sends the messages through the SendGrid API.
 4. Once the provider accepts the message (HTTP 202), the record updates to `reminder_sent = true`.
 - **Alternative and Error Flows:**
 - **A1: Email Provider Failure:** If the provider call times out or returns an error, the error is logged, and the appointment remains unflagged for processing in the subsequent cron cycle.
-

CU-11: Personal Dashboard Management

- **ID:** CU-11
- **Name:** View Personal Appointments and History
- **Primary Actor:** Student or Tutor

- **Secondary Actors:** Database (PostgreSQL)
 - **Trigger:** The user opens the `/dashboard` route.
 - **Preconditions:** The user holds a valid, unexpired JWT session.
 - **Postconditions:**
 - **Success:** Upcoming sessions and historical appointments are displayed.
 - **Failure:** The user is redirected to the login view if the session token is expired.
 - **Business Rules:** Appointments automatically move from upcoming to historical status once their end timestamp has passed.
 - **Main Success Scenario (Happy Path):**
 1. The user navigates to their dashboard.
 2. The frontend requests `GET /api/appointments/me` with the session cookie (`credentials: 'include'`).
 3. The backend returns appointments categorized into upcoming and completed sessions.
 4. The interface displays appointment cards with direct meeting links and cancellation options.
 - **Alternative and Error Flows:**
 - **A1: Expired Token:** If the JWT signature is expired, the backend returns an HTTP 401 Unauthorized, and the web client redirects to the login view (the server clears the expired cookie).
-

CU-12: Post-Session Notes and Rating

- **ID:** CU-12
- **Name:** Submit Tutoring Notes and Feedback
- **Primary Actor:** Tutor or Student
- **Secondary Actors:** Database (PostgreSQL)
- **Trigger:** The user clicks "Add Notes / Review" on a completed appointment.
- **Preconditions:** The appointment must be in the user's history and marked as completed.
- **Postconditions:**
 - **Success:** Private tutor notes and student ratings are saved to PostgreSQL.
 - **Failure:** Data is not persisted, and input/network errors are reported.
- **Business Rules:** Tutor notes are visible only to the participating student and tutor; students can submit a rating (1 to 5 stars) only once per completed session.
- **Main Success Scenario (Happy Path):**
 1. Once a session has concluded, the user opens the historical session details.
 2. The tutor documents discussion topics and recommended follow-up study resources.
 3. The student submits a numerical rating (1–5) and optional written feedback.
 4. The backend verifies inputs and updates the appointment record in the database.
- **Alternative and Error Flows:**

- **A1: Premature Feedback Attempt:** If a user attempts to submit feedback before the session end time, inputs remain locked: *"Ratings and notes are accessible only after session completion."*
-

CU-13: Account Moderation and Suspension (Admin)

- **ID:** CU-13
 - **Name:** Moderate and Suspend Problematic Accounts
 - **Primary Actor:** University Administrator (Admin)
 - **Secondary Actors:** Database (PostgreSQL)
 - **Trigger:** The admin clicks "Suspend Account" in the user management console.
 - **Preconditions:** The session token must contain verified **ADMIN** role privileges.
 - **Postconditions:**
 - **Success:** The account status updates to **SUSPENDED**, active sessions are revoked, future appointments are canceled and their slots are released.
 - **Failure:** The account remains unchanged if permissions are invalid.
 - **Business Rules:** An administrator cannot suspend their own admin account. Suspended accounts are rejected on every authenticated request because the server checks the account status in the database, which invalidates any JWT already issued.
 - **Main Success Scenario (Happy Path):**
 1. The admin accesses the user directory in the admin panel.
 2. The admin locates the reported account and clicks "Suspend Account".
 3. The admin provides an administrative reason and confirms.
 4. The server validates administrative role claims from the JWT.
 5. A database transaction updates the user status to **SUSPENDED**, cancels all their pending future bookings and releases the affected slots (back to **AVAILABLE** for a suspended student's appointments, **BLOCKED** for a suspended tutor's). The counterparties are notified by email after the commit.
 6. The management view updates the user status badge immediately.
 - **Alternative and Error Flows:**
 - **A1: Unauthorized Execution:** If a student or tutor sends a request to the admin endpoint, the server returns an HTTP 403 Forbidden.
-

CU-14: System Metrics and Audit Log Review (Admin)

- **ID:** CU-14
- **Name:** Inspect Global Platform Analytics and Logs
- **Primary Actor:** University Administrator (Admin)
- **Secondary Actors:** Database (PostgreSQL)
- **Trigger:** The admin navigates to the "System Metrics & Auditing" view.

- **Preconditions:** Valid authentication with an Administrator account.
 - **Postconditions:**
 - **Success:** High-level operational metrics (booking totals, active tutors, cancellation rates) and audit trails are displayed.
 - **Failure:** Access is denied for non-admin accounts.
 - **Business Rules:** Audit logs are append-only and cannot be modified or deleted.
 - **Main Success Scenario (Happy Path):**
 1. The admin opens the analytics dashboard.
 2. The backend executes aggregation queries across users, availability slots, and appointment tables.
 3. Calculated metrics (completed sessions, cancellation ratios, active tutor capacity) are returned as structured JSON.
 4. The interface renders overview charts and summary tables.
-

4. Non-Functional Requirements (NFRs) and Testing Methodology

As mandated by project guidelines, non-functional requirements are categorized with specific verification methods:

NFR-01 · Performance

- **Requirement:** API responses for directory queries and calendar availability must complete in under **200 ms** (95th percentile) under a sustained load of 50 concurrent virtual users, consistent with goal G5.
- **Testing Method:** Automated load and stress testing will be conducted using **Artillery and k6**, targeting backend Express endpoints to benchmark latency and throughput under peak loads.

NFR-02 · Security

- **Requirement:** Passwords must be hashed using **bcrypt** (work factor ≥ 10). Protected endpoints must enforce signed **JWT** verification, with the token held in an **HttpOnly, Secure** cookie and **SameSite**/CSRF protection configured for the cross-site Vercel/Render deployment (ADR 4). Database interactions must use parameterized queries to prevent SQL Injection; Cross-Site Scripting (XSS) is mitigated separately through output encoding of user-supplied text, input validation and a Content-Security-Policy header.
- **Testing Method:** Static dependency and code vulnerability scanning via **SonarQube and npm audit**, accompanied by dynamic penetration testing (DAST) using **OWASP ZAP**, plus a manual check of cookie flags and CSRF protection.

NFR-03 · Reliability & Concurrency

- **Requirement:** The platform must guarantee ACID transaction isolation in PostgreSQL during slot bookings to prevent race conditions and double bookings under simultaneous student requests.
- **Testing Method:** Automated integration testing using **Jest and Supertest** executing concurrent requests against the same slot identifier to verify that exactly one request succeeds (HTTP 201) while simultaneous attempts receive an HTTP 409 Conflict.

NFR-04 · Maintainability

- **Requirement:** The backend architecture must follow a decoupled layered pattern (Routes, Controllers, Services, and Data Access/Repositories), maintaining at least **70% unit test code coverage**.
- **Testing Method:** Enforced static linting through **ESLint** alongside continuous automated code coverage reports generated by Jest and verified within **GitHub Actions** workflows.